Computer Architecture
CS 622 / EE 520
Spring 2026

Lecture 13

Shahid Masud

Topics

- Review Superscalar, Multi-Issue, and ILP
- VLIW
- Solve two examples
  - Step through Reorder Buffer
- Branch Prediction Buffer Cache

Quiz 3
Next Lecture

Midterm
Week 9

Assignment
2 Soon

Further Readings

Reading Notes

Readings:
Chapter 3 of course textbook

Hardware Based Speculation – Using Reorder Buffer

Recap – Done Last time

Tomasulo with Speculation

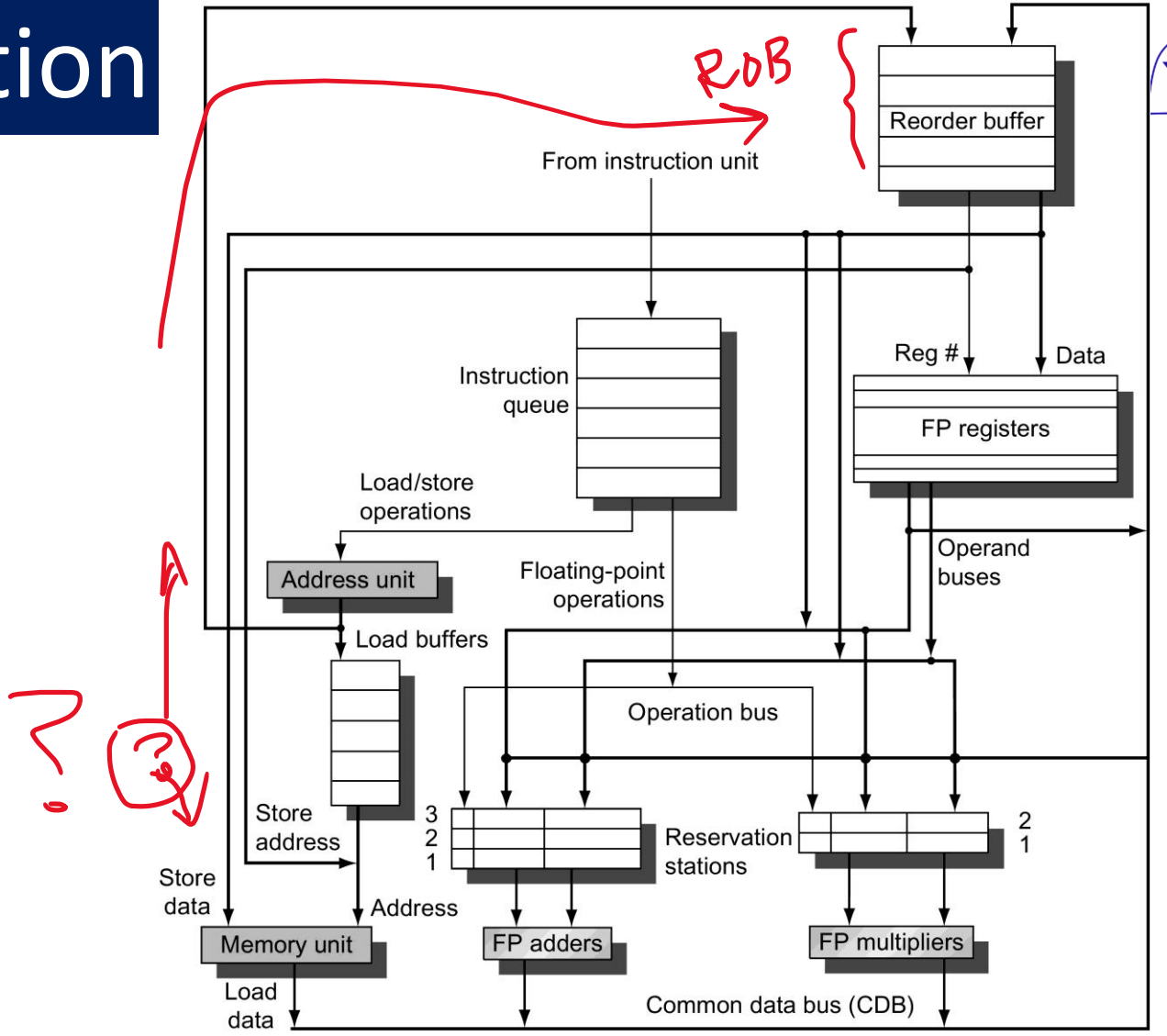
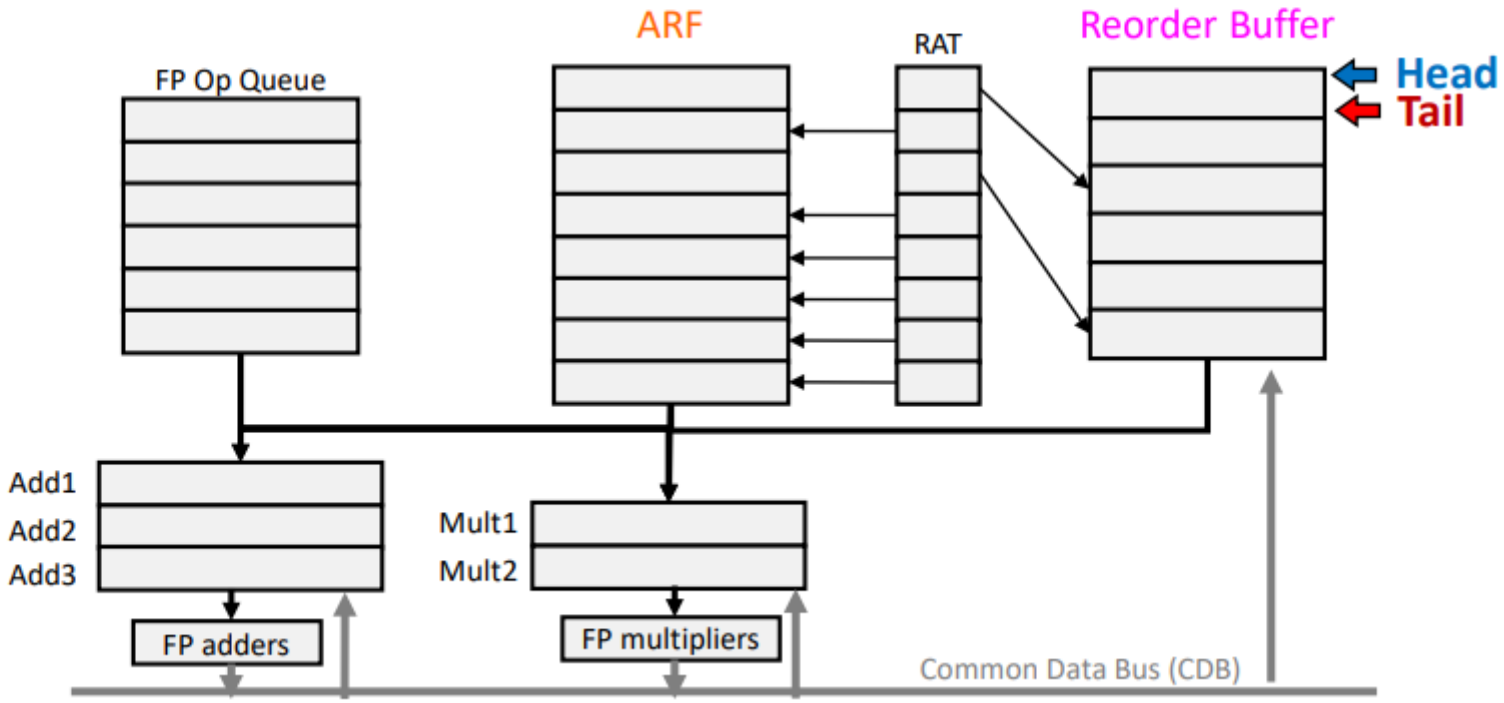


Figure 3.15 The basic structure of a FP unit using Tomasulo's algorithm and extended to handle speculation. Comparing this to Figure 3.10 on page 198, which implemented Tomasulo's algorithm, we can see that the major change is the addition of the ROB and the elimination of the store buffer, whose function is integrated into the ROB. This mechanism can be extended to allow multiple issues per clock by making the CDB wider to allow for multiple completions per clock.

Tomasulo with ROB Organization

- Re-Order buffer is based on Tomasulo
- Just renamed FP register to **ARF** (Architected Register File)
- Add **Re-Order Buffer** for out-of-order results
  - Buffer is managed with two pointers (head & tail)
- RAT (Register Alias Table) keeps the register renaming info.



4 Steps for Speculative Tomasulo Algorithm

- Instruction Issue
- Execution
  - Start
  - Complete
- Write Back
- Commit

Example page 212 (come back later after examples)

Once an instruction commits, its entry in the ROB is reclaimed, and the register or memory destination is updated, eliminating the need for the ROB entry. If the ROB fills, we simply stop issuing instructions until an entry is made free. Now let's examine how this scheme would work with the same example we used for Tomasulo's algorithm.

Example Assume the same latencies for the floating-point functional units as in earlier examples: add is 2 clock cycles, multiply is 6 clock cycles, and divide is 12 clock cycles. Using the following code segment, the same one we used to generate Figure 3.12, show what the status tables look like when the `fmul.d` is ready to go to commit.

```
fld     f6,32(x2)
fld     f2,44(x3)
fmul.d  f0,f2,f4
fsub.d  f8,f2,f6
fdiv.d  f0,f0,f6
fadd.d  f6,f8,f2
```

Answer Figure 3.16 shows the result in the three tables. Notice that although the `fsub.d` instruction has completed execution, it does not commit until the `fmul.d` commits. The reservation stations and register status field contain the same basic information that they did for Tomasulo's algorithm (see page 200 for a description of those fields). The differences are that reservation station numbers are replaced with ROB entry numbers in the Qj and Qk fields, as well as in the register status fields, and we added the Dest field to the reservation stations. The Dest field designates the ROB entry that is the destination for the result produced by this reservation station entry.

Solution

	Reorder buffer					
Entry	Busy	Instruction		State	Destination	Value
1	No	`fld`	`f6,32(x2)`	Commit	`f6`	`Mem[32 + Regs[x2]]`
2	No	`fld`	`f2,44(x3)`	Commit	`f2`	`Mem[44 + Regs[x3]]`
3	Yes	`fmul.d`	`f0,f2,f4`	Write result	`f0`	#2 × `Regs[f4]`
4	Yes	`fsub.d`	`f8,f2,f6`	Write result	`f8`	#2 – #1
5	Yes	`fdiv.d`	`f0,f0,f6`	Execute	`f0`	
6	Yes	`fadd.d`	`f6,f8,f2`	Write result	`f6`	#4 + #2

	Reservation stations							
Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	A
Load1	No							
Load2	No							
Add1	No							
Add2	No							
Add3	No							
Mult1	No	`fmul.d`	`Mem[44 + Regs[x3]]`	`Regs[f4]`			#3	
Mult2	Yes	`fdiv.d`		`Mem[32 + Regs[x2]]`	#3		#5	

	FP register status									
Field	f0	f1	f2	f3	f4	f5	f6	f7	f8	f10
Reorder #	3						6		4	5
Busy	Yes	No	No	No	No	No	Yes	…	Yes	Yes

Figure 3.16 At the time the `fmul.d` is ready to commit, only the two `fld` instructions have committed, although several others have completed execution. The `fmul.d` is at the head of the ROB, and the two `fld` instructions are there only to ease understanding. The `fsub.d` and `fadd.d` instructions will not commit until the `fmul.d` instruction commits, although the results of the instructions are available and can be used as sources for other instructions. The `fdiv.d` is in execution, but has not completed solely because of its longer latency than that of `fmul.d`. The Value column indicates the value being held; the format #X is used to refer to a value field of ROB entry X. Reorder buffers 1 and 2 are actually completed but are shown for informational purposes. We do not show the entries for the load/store queue, but these entries are kept in order.

Solution - contd

Reorder buffer

Entry	Busy	Instruction		State	Destination	Value
1	No	`fld`	`f0,0(x1)`	Commit	`f0`	`Mem[0 + Regs[x1]]`
2	No	`fmul.d`	`f4,f0,f2`	Commit	`f4`	#1 × `Regs[f2]`
3	Yes	`fsd`	`f4,0(x1)`	Write result	`0 + Regs[x1]`	#2
4	Yes	`addi`	`x1,x1,−8`	Write result	`x1`	`Regs[x1] − 8`
5	Yes	`bne`	`x1,x2,Loop`	Write result		
6	Yes	`fld`	`f0,0(x1)`	Write result	`f0`	`Mem[#4]`
7	Yes	`fmul.d`	`f4,f0,f2`	Write result	`f4`	#6 × `Regs[f2]`
8	Yes	`fsd`	`f4,0(x1)`	Write result	0 + #4	#7
9	Yes	`addi`	`x1,x1,−8`	Write result	`x1`	#4 − 8
10	Yes	`bne`	`x1,x2,Loop`	Write result		

FP register status

Field	f0	f1	f2	f3	f4	F5	f6	F7	f8
Reorder #	6								
Busy	Yes	No	No	No	Yes	No	No	…	No

Figure 3.17 Only the `fld` and `fmul.d` instructions have committed, although all the others have completed execution. Thus no reservation stations are busy and none are shown. The remaining instructions will be committed as quickly as possible. The first two reorder buffers are empty, but are shown for completeness.

Example 1 of ROB

Example 2 of ROB

Exceptions in Speculative Execution

Register Renaming and Exceptions in ROB

- **Register Renaming**

In the presence of a ROB, the renaming function of the reservation stations is replaced by the ROB. A result is tagged using the ROB entry number rather than using the reservation station number.

- **Exceptions**

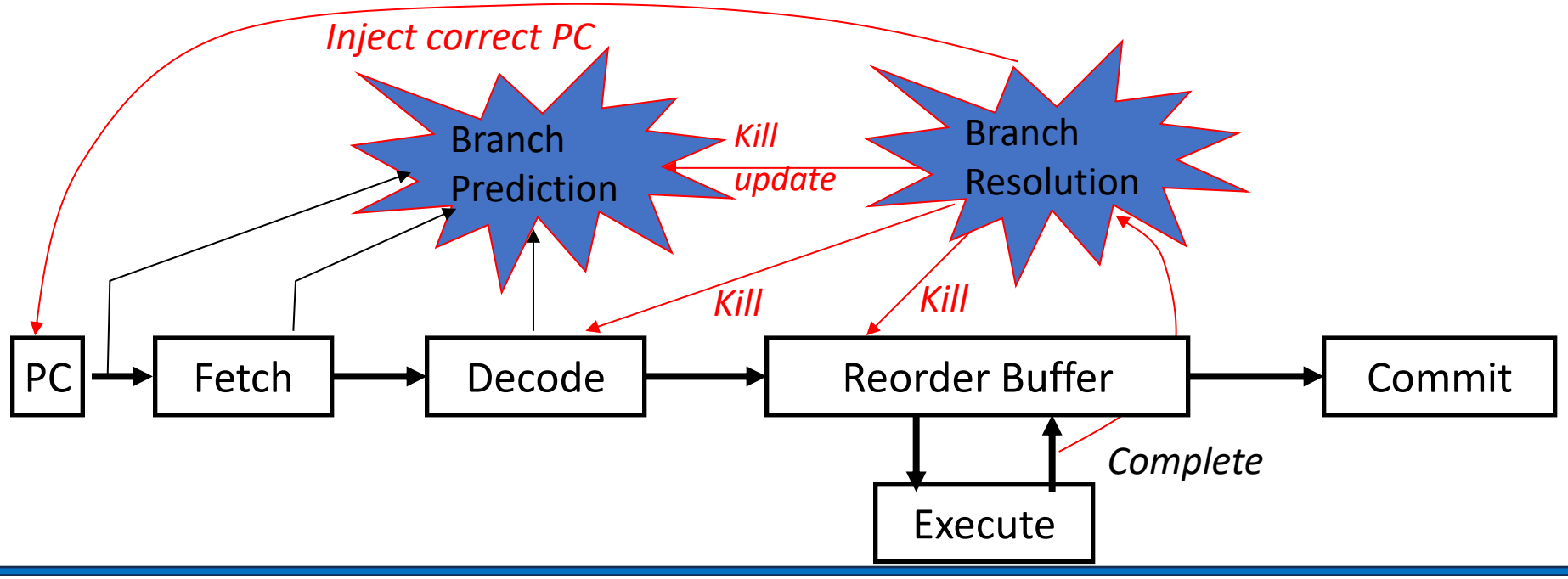
The use of a ROB with in-order instruction commit provides precise exceptions, in addition to supporting speculative execution.

- **Exceptions are handled**

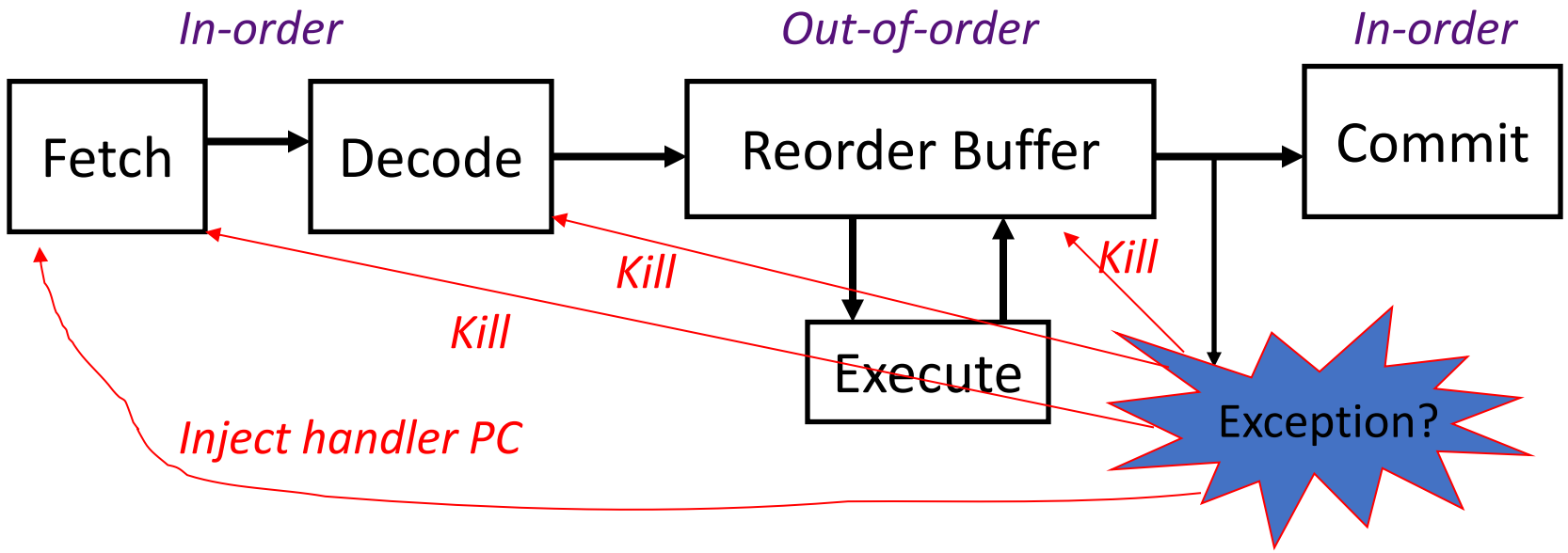
By not recognizing the exception until it is ready to commit. If a speculated instruction raises an exception, the exception is recorded in the ROB. If a branch misprediction arises and the instruction should not have been executed, the exception is flushed along with the instruction when the ROB is cleared. If the instruction reaches the head of the ROB, then we know it is no longer speculative, and the exception should really be taken.

Handling Incorrect Speculation

- Instructions following mispredicted branch, i.e. those in decode/issue buffers & RSs, are invalidated
- ROB entries of these instructions are deallocated
- Restart fetch at correct branch successor



Handling Precise Exception / Interrupt



- Instructions fetched and decoded into ROB in-order
- Execution is out-of-order → OOO completion
- Commit (write-back to architectural state, i.e., register file and memory) is in-order

Steps 1 – 2 in Dynamic Scheduler with Speculation

Apart from the three steps in the dynamic scheduler, an **additional step of commit** is introduced when there is support for speculation. **Total four steps** in a dynamic scheduler with speculation are below:

- **Issue**
  - Get next instruction from FIFO queue
  - If available RS entry and ROB entry, issue the instruction
  - If a RS or ROB entry is not available, it becomes a structural hazard and the instruction stalls
  - If an earlier instruction is not issued, then subsequent instructions cannot be issued
  - If operands are available read them and store them in the reservation station\
  - Operands can be read from the register file (already committed instructions) or from the ROB (executed but not yet committed)
  - Operands dependent on an earlier instruction are identified by their corresponding ROB tag in the Q field and such instructions will have to stall till data becomes available
- **Execute**
  - The CDB broadcasts data with an ROB tag and when an operand becomes available, store it in any reservation station waiting for it
  - When all operands are ready, issue the instruction for execution
  - Free the reservation station entry once the instruction moves into execution
  - Loads and store are maintained in program order through the effective address
  - Even with pending branches, instructions are speculatively executed

Steps 3 – 4 in Dynamic Scheduler with Speculation

- **Write result**
  - Write result on CDB into reservation stations and ROB
- **Commit**
  - An instruction is committed when it reaches the head of the ROB and its result is present in the buffer
  - If it is an ALU or Load instruction, update the register with the result and remove the instruction from the ROB
  - Committing a store is similar except that memory is updated rather than a result register
  - If a branch with incorrect prediction reaches the head of the ROB, it indicates that the speculation was wrong
  - ROB is flushed and execution is restarted at the correct successor of the branch
  - If the branch was correctly predicted, the branch is removed from ROB
  - Once an instruction commits, its entry in the ROB is reclaimed
  - If the ROB fills, we simply stop issuing instructions until an entry is made free.

ROB Example 1

Let us consider the sequence of instructions below.
Assume the latencies for the Add, Mul and Div instructions to be 2, 6 and 12 respectively.

Shows how the status will look like when the MUL.D is ready to go to commit

```
L.D     F6,32(R2)
L.D     F2,44(R3)
MUL.D F0,F2,F4
SUB.D F8,F6,F2
DIV.D F10,F0,F6
ADD.D F6,F8,F2
```

Solution

Figure shows how the status tables will look like when the MUL.D is ready to go to commit.
Observe that though the Sub and Add instructions have written the result, they have not yet committed.
This is because the Mul instruction has not yet committed.
The Destination field designates the ROB entry that is the destination for the result produced by a particular reservation station entry.
The complete schedule is given in following Table.
Note that the instructions are issued in-order, executed out-of-order, completed out-of-order, but committed in-order.

Instruction	Issues at	Executes at	Writes result at	Commits at
L.D F6,32(R2)	1	2-3	4	5
L.D F2,44(R3)	2	3-4	5	6
MUL.D F0,F2,F4	3	6-11	12	13
SUB.D F8,F6,F2	4	6-7	8	14
DIV.D F10,F0,F6	5	13-24	25	26
ADD.D F6,F8,F2	6	9-10	11	27



'In Order' Commit, Same as Instructions

Solution - contd

add is 2 clock cycles, multiply is 6 clock cycles, and divide is 12 clock cycles.

Reorder buffer

Entry	Busy	Instruction		State	Destination	Value
1	No	L.D	F6,32(R2)	Commit	F6	Mem[32 + Regs[R2]]
2	No	L.D	F2,44(R3)	Commit	F2	Mem[44 + Regs[R3]]
3	Yes	MUL.D	F0,F2,F4	Write result	F0	#2 × Regs[F4]
4	Yes	SUB.D	F8,F2,F6	Write result	F8	#2 – #1
5	Yes	DIV.D	F10,F0,F6	Execute	F10	
6	Yes	ADD.D	F6,F8,F2	Write result	F6	#4 + #2

Reservation stations

Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	A
Load1	No							
Load2	No							
Add1	No							
Add2	No							
Add3	No							
Mult1	No	MUL.D	Mem[44 + Regs[R3]]	Regs[F4]			#3	
Mult2	Yes	DIV.D		Mem[32 + Regs[R2]]	#3		#5	

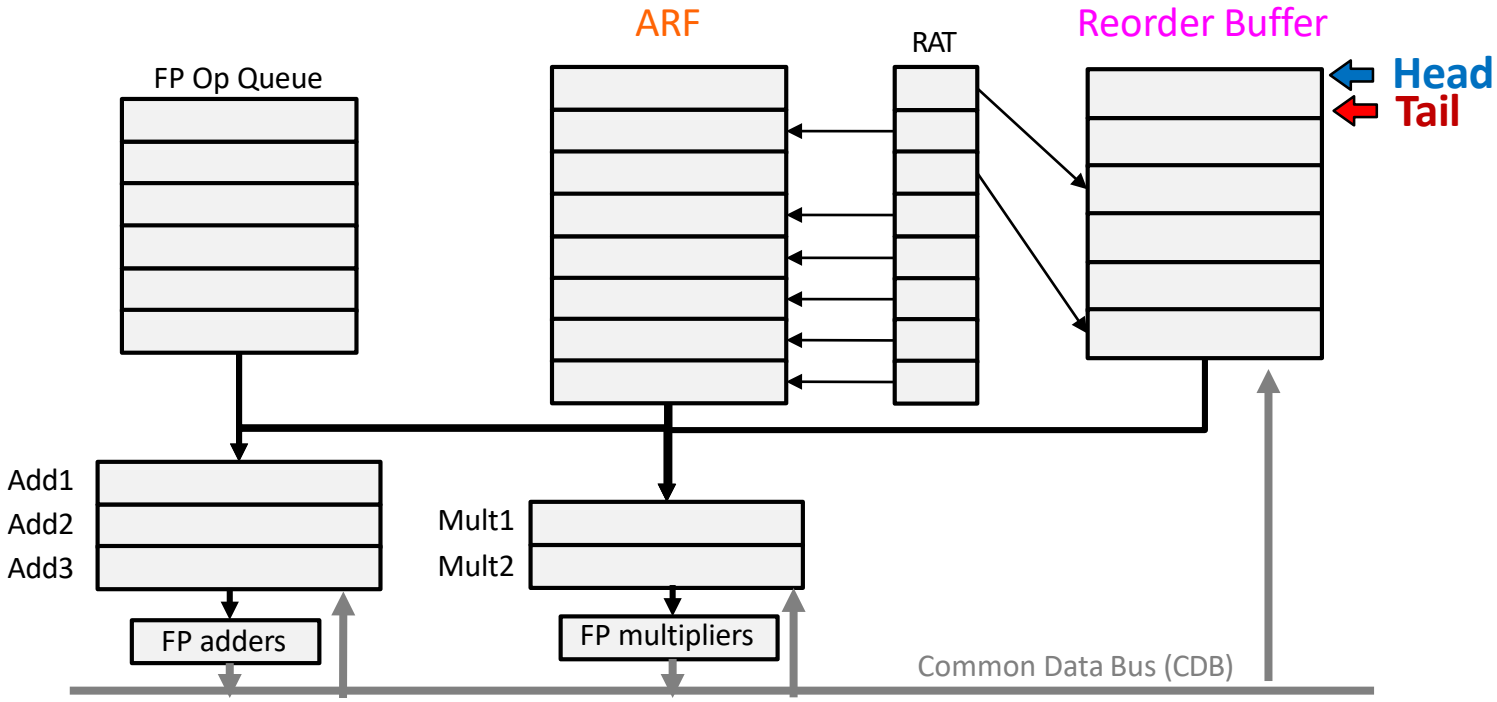
FP register status

Field	F0	F1	F2	F3	F4	F5	F6	F7	F8	F10
Reorder #	3						6		4	5
Busy	Yes	No	No	No	No	No	Yes	. . .	Yes	Yes

Figure 3.12 At the time the MUL.D is ready to commit, only the two L.D instructions have committed, although several others have completed execution. The MUL.D is at the head of the ROB, and the two L.D instructions are there only to ease understanding. The SUB.D and ADD.D instructions will not commit until the MUL.D instruction commits, although the results of the instructions are available and can be used as sources for other instructions. The DIV.D is in execution, but has not completed solely due to its longer latency than MUL.D. The Value column indicates the value being held; the format #X is used to refer to a value field of ROB entry X. Reorder buffers 1 and 2 are actually completed but are shown for informational purposes. We do not show the entries for the load/store queue, but these entries are kept in order.

Tomasulo w/ ROB Organization

- Re-Order buffer is based on Tomasulo
- Just renamed FP register to **ARF** (Architected Register File)
- Add **Re-Order Buffer** for out-of-order results
  - Buffer is managed with two pointers (head & tail)
- RAT (Register Alias Table) keeps the register renaming info.



Four Stages of Tomasulo w/ ROB

Stage1: Issue

- Read an instruction from the buffer
- Check if resources available (RS entry & ROB entry)
- Read RAT

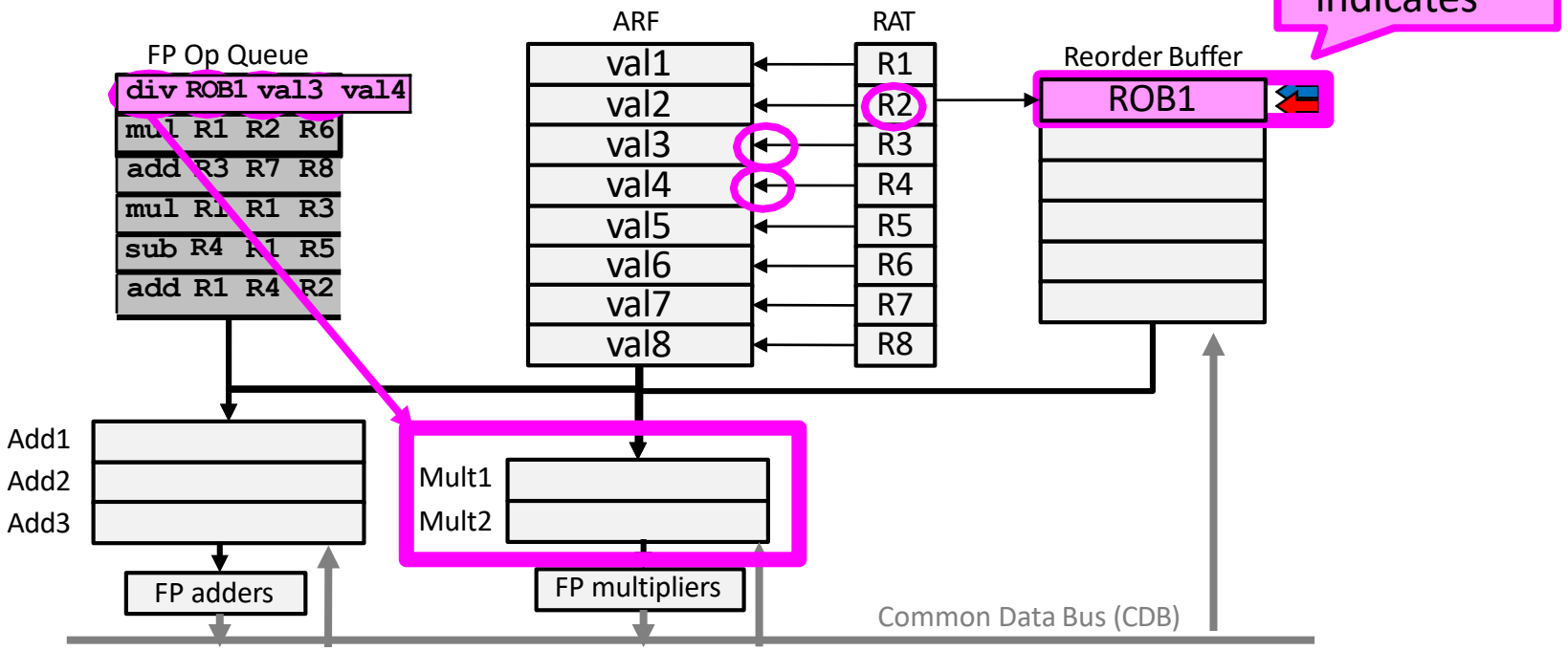
  Sources: read (available) sources

  Destination: update RAT

- Write to RS and ROB (the tail of ROB) & update the tail of ROB

Stall if any target resource is unavailable

Where the tail pointer indicates

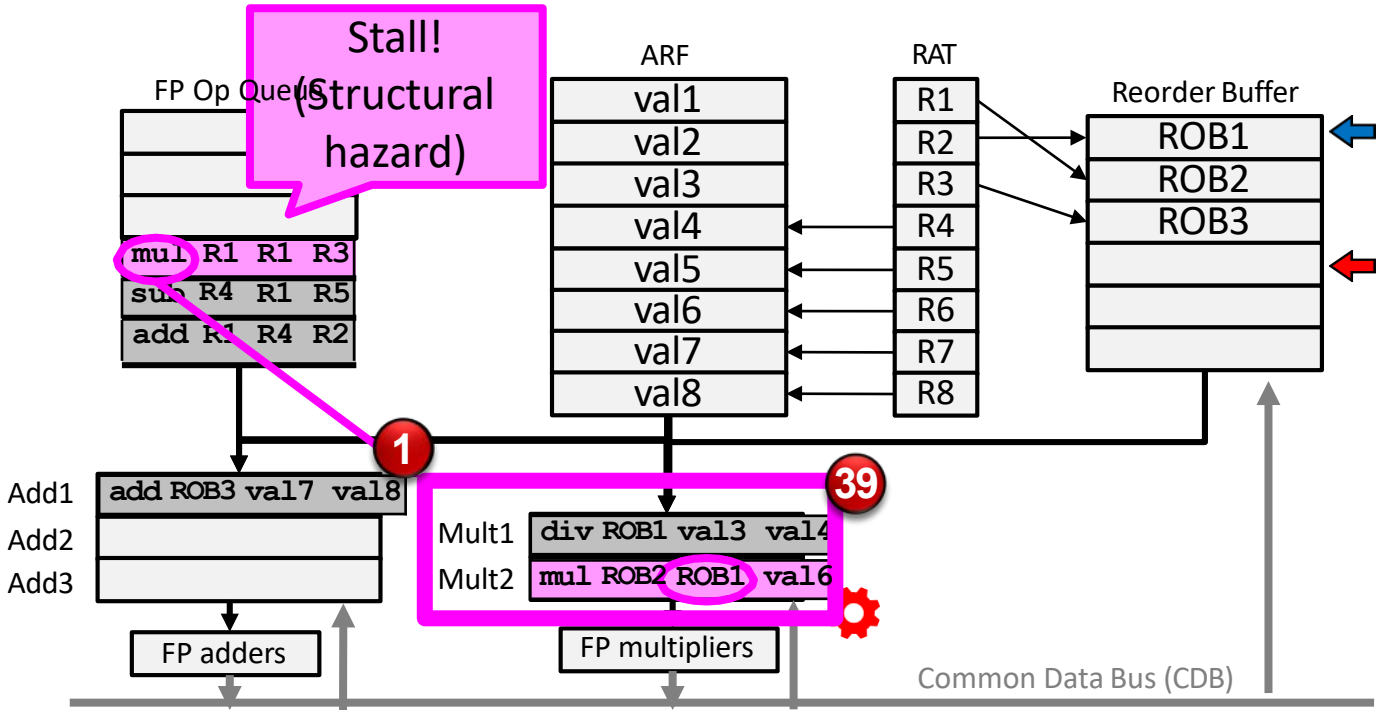


Four Stages of Tomasulo w/ ROB

Stage2: Exec

➔ Different behaviors based on the target instruction status

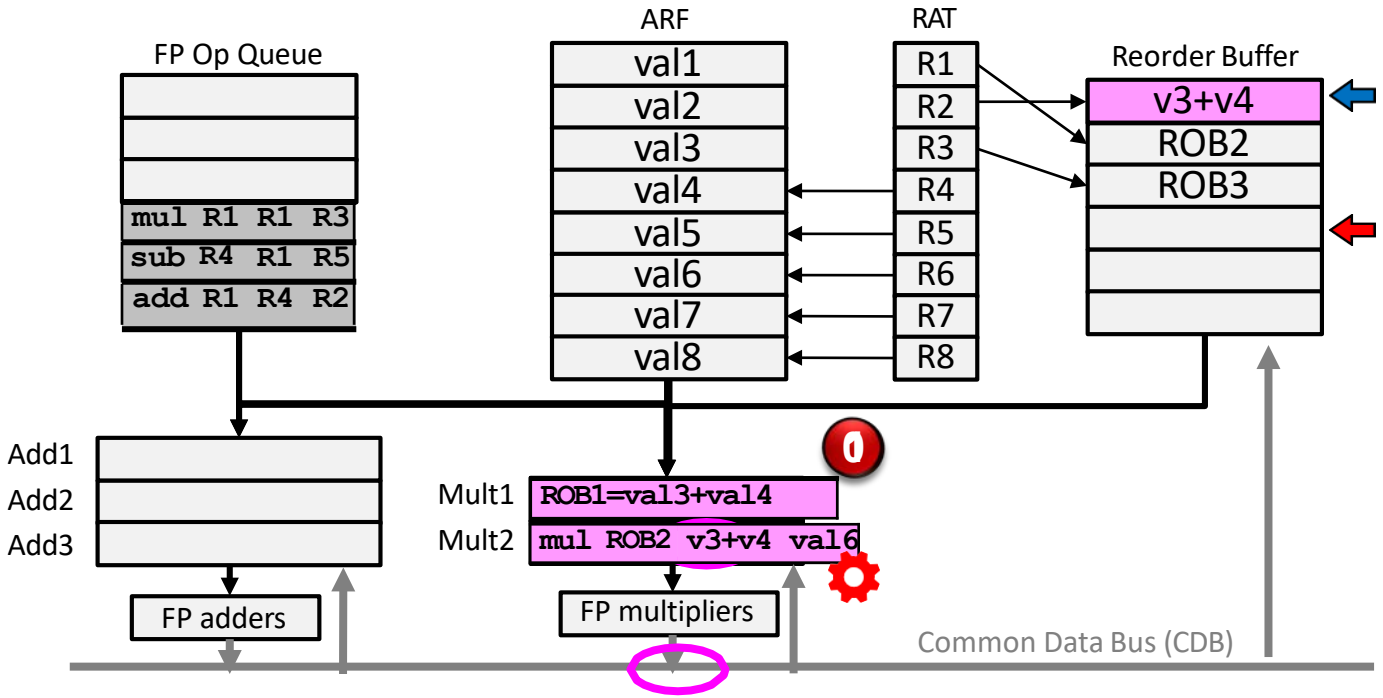
- For “a not-ready instruction” → Wait for all operands to arrive
- For “a new instruction” → Compete to use the functional unit
- For “an all-ready instruction” → Execute!



Four Stages of Tomasulo w/ ROB

Stage3: Write Result

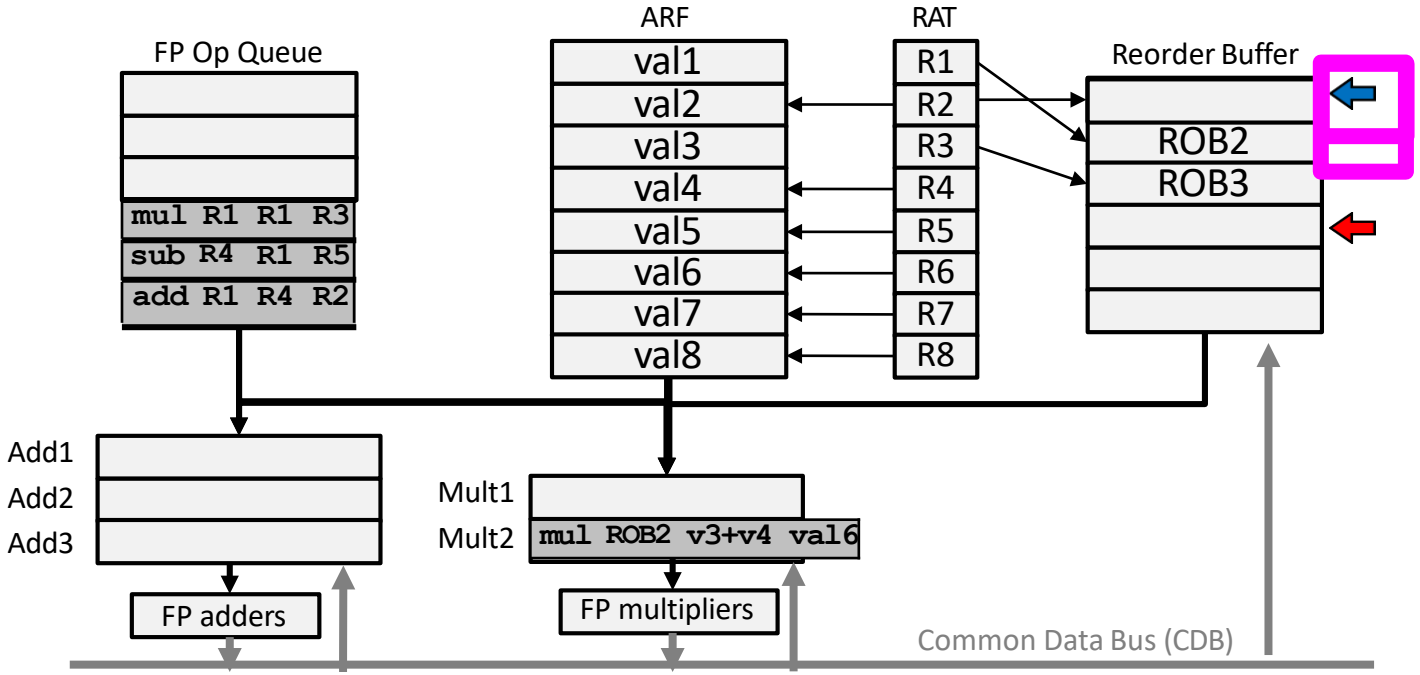
- Broadcast result on CDB
  - (any dependents will grab the value)
- Write result back to your ROB entry
  - The ARF holds the “official” register state, which we will only update in the program order
  - Mark ready/finished bit in ROB (note that this instruction has completed execution)
- Reservation station can be freed



Four Stages of Tomasulo w/ ROB

Stage4: Commit

- When the oldest instruction is completed in the ROB
  - i.e., ROB's head points to it
- Write the result (if ready/finished bit is set)
- Advance ROB's head to the next instruction
- This is what the outside world sees
  - And it's all in-order



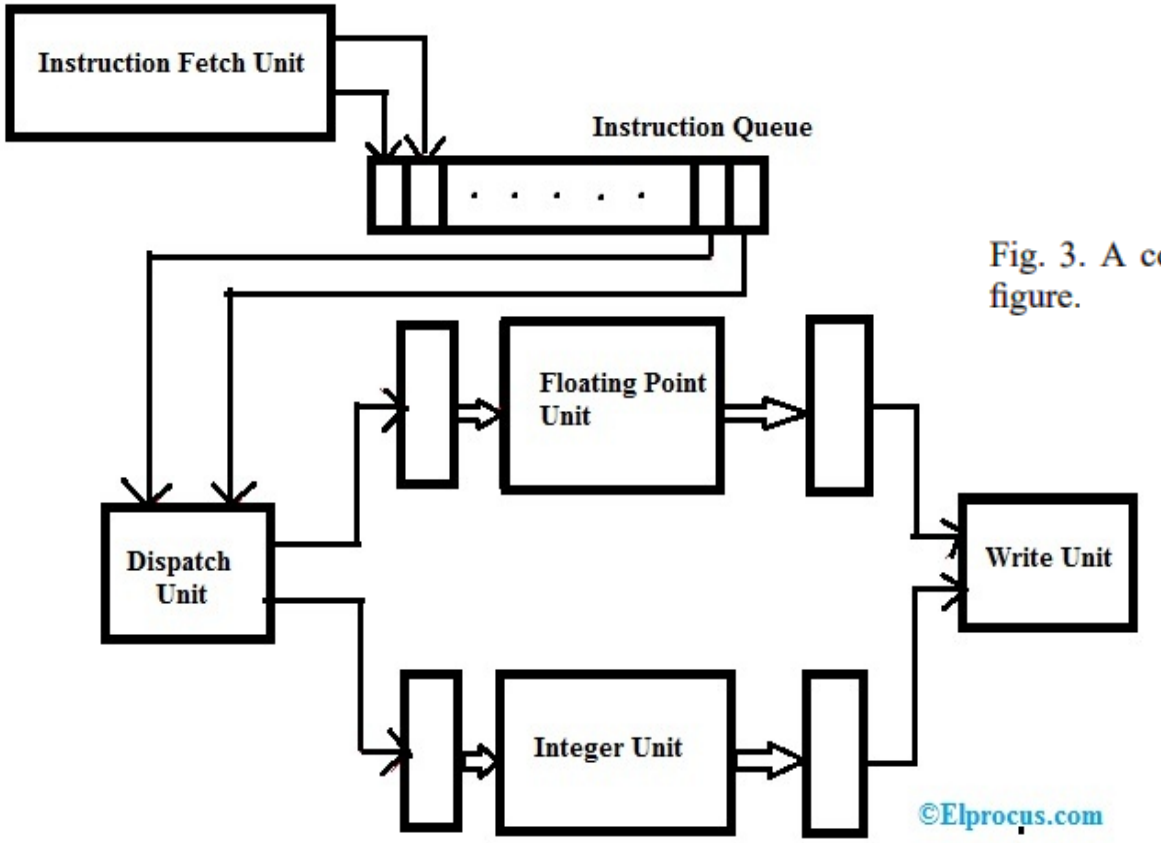
Exploiting ILP Using Superscalar, Multiple Issue and Static Scheduling

Superscalar Architecture

Superscalar processors issue more than one instruction per clock cycle. Unlike VLIW processors, they check for resource conflicts on the fly to determine what combinations of instructions can be issued at each step. Superscalar architectures dominate desktop and server architectures. Superscalar processors are not as common in the embedded world as in the desktop/server world. Embedded computing architectures are more likely to be judged by metrics such as operations per watt rather than raw performance.

A surprising number of embedded processors do, however, make use of superscalar instruction issue, though not as aggressively as do high-end servers. The embedded Pentium processor is a two-issue, in-order processor. It has two pipes: one for any integer operation and another for simple integer operations. We saw in Section 2.3.1 that other embedded processors also use superscalar techniques.

Superscalar Example



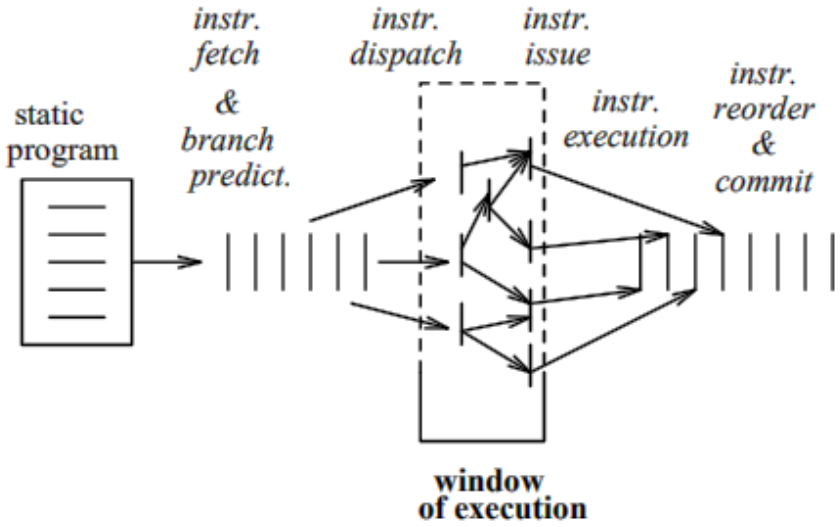


Fig. 3. A conceptual figure of superscalar execution. Processing phases are listed across the top of the figure.

Multiple Instructions Issued

The ALU and data transfer instructions are issued at the same time.

Instruction type	Pipe stages							
ALU or branch instruction	IF	ID	EX	MEM	WB			
Load or store instruction	IF	ID	EX	MEM	WB			
ALU or branch instruction		IF	ID	EX	MEM	WB		
Load or store instruction		IF	ID	EX	MEM	WB		
ALU or branch instruction			IF	ID	EX	MEM	WB	
Load or store instruction			IF	ID	EX	MEM	WB	
ALU or branch instruction				IF	ID	EX	MEM	WB
Load or store instruction				IF	ID	EX	MEM	WB

Superscalar Processor Types

The two types of superscalar processors issue varying numbers of instructions per clock and use in-order execution if they are statically scheduled or out-of-order execution if they are dynamically scheduled.

VLIW processors, in contrast, issue a fixed number of instructions formatted either as one large instruction or as a fixed instruction packet with the parallelism among instructions explicitly indicated by the instruction. VLIW processors are inherently statically scheduled by the compiler. When Intel and HP created the IA-64 architecture, described in Appendix H, they also introduced the name EPIC (explicitly parallel instruction computer) for this architectural style.

Although statically scheduled superscalars issue a varying rather than a fixed number of instructions per clock, they are actually closer in concept to VLIWs because both approaches rely on the compiler to schedule code for the processor. Because of the diminishing advantages of a statically scheduled superscalar as the issue width grows, statically scheduled superscalars are used primarily for narrow issue widths, normally just two instructions. Beyond that width, most designers choose to implement either a VLIW or a dynamically scheduled superscalar. Because of the similarities in hardware and required compiler technology, we focus on VLIWs in this section, and we will see them again in Chapter 7. The insights of this section are easily extrapolated to a statically scheduled superscalar.

ILP Differences

Very Important Terms

- Dynamic Scheduling → Out-of-order Execution
- Speculation → In-order Commit
- Superscalar → Multiple Issue

Techniques	Goals	Implementation	Addressing	Approaches
Dynamic Scheduling	Out-of-order execution	Reservation Stations, Load/Store Buffer and CDB	Data hazards (RAW, WAW, WAR)	Register renaming
Speculation	In-order commit	Branch Prediction + Reorder Buffer	Control hazards	Prediction and misprediction recovery
Superscalar /VLIW	Multiple issue	Software and Hardware	To Increase CPI	By compiler or hardware

5 Primary Approaches for Multi-issue Processor

Common name	Issue structure	Hazard detection	Scheduling	Distinguishing characteristic	Examples
Superscalar (static)	Dynamic	Hardware	Static	In-order execution	Mostly in the embedded space: MIPS and ARM, including the Cortex-A53
Superscalar (dynamic)	Dynamic	Hardware	Dynamic	Some out-of-order execution, but no speculation	None at the present
Superscalar (speculative)	Dynamic	Hardware	Dynamic with speculation	Out-of-order execution with speculation	Intel Core i3, i5, i7; AMD Phenom; IBM Power 7
VLIW/LIW	Static	Primarily software	Static	All hazards determined and indicated by compiler (often implicitly)	Most examples are in signal processing, such as the TI C6x
EPIC	Primarily static	Primarily software	Mostly static	All hazards determined and indicated explicitly by the compiler	Itanium

Figure 3.19 The five primary approaches in use for multiple-issue processors and the primary characteristics that distinguish them. This chapter has focused on the hardware-intensive techniques, which are all some form of superscalar. Appendix H focuses on compiler-based approaches. The EPIC approach, as embodied in the IA-64 architecture, extends many of the concepts of the early VLIW approaches, providing a blend of static and dynamic approaches.

Superpipelining vs Multiple Issue

Superpipelining

Increase the depth of the pipeline to increase the clock rate

The more stages in the pipeline, the more forwarding/hazard hardware needed and the more pipeline latch overhead (i.e., the pipeline latch accounts for a larger and larger percentage of the clock cycle time)

Multiple-Issue

Fetch (and execute) more than one instructions at one time (expand every pipeline stage to accommodate multiple instructions)

Multiple Issue

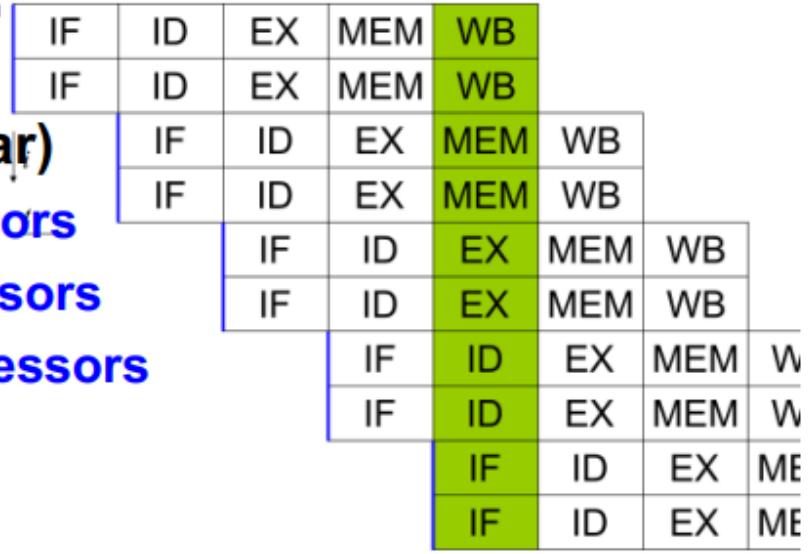
- **"Flynn bottleneck"**
  - **single issue performance limit is CPI = IPC = 1**
  - **hazards + overhead ⇒ CPI >= 1 (IPC <= 1)**
  - **diminishing returns from superpipelining [Hrishikesh paper!]**
- **Solution: issue multiple instructions per cycle**

	1	2	3	4	5	6	7
inst0	F	D	X	M	W		
inst1	F	D	X	M	W		
inst2		F	D	X	M	W	
inst3		F	D	X	M	W	

- **1st superscalar: IBM America → RS/6000 → POWER1**

Multiple Issue, Superscalar and VLIW

- Issue multiple instructions in one cycle
- Three major types (VLIW and superscalar)
  - Statically scheduled superscalar processors
  - VLIW (very long instruction word) processors
  - Dynamically scheduled superscalar processors
- Superscalar
  - Variable # of instr per cycle
  - In-order execution for static superscalar
  - Out-of-order execution for dynamic superscalar
- VLIW
  - Issue a fixed number of instructions formatted either as one large instruction or as a fixed instruction packet with the parallel- ism among instructions explicitly indicated by the instruction.
  - Inherently statically scheduled by the compiler
  - Intel/HP IA-64 architecture, named EPIC—explicitly parallel instruction computer



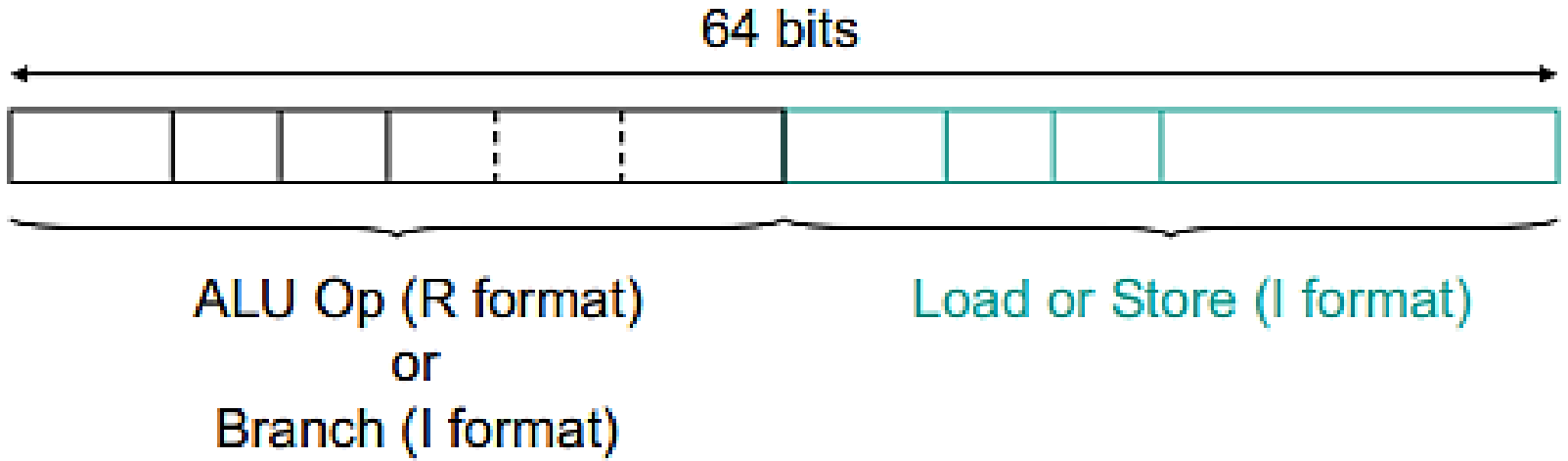
Goal of Multiple-Issue Processors

The goal of the *multiple-issue processors*, discussed in the next few sections, is to allow multiple instructions to issue in a clock cycle. Multiple-issue processors come in three major flavors:

1. Statically scheduled superscalar processors
2. VLIW (very long instruction word) processors
3. Dynamically scheduled superscalar processors

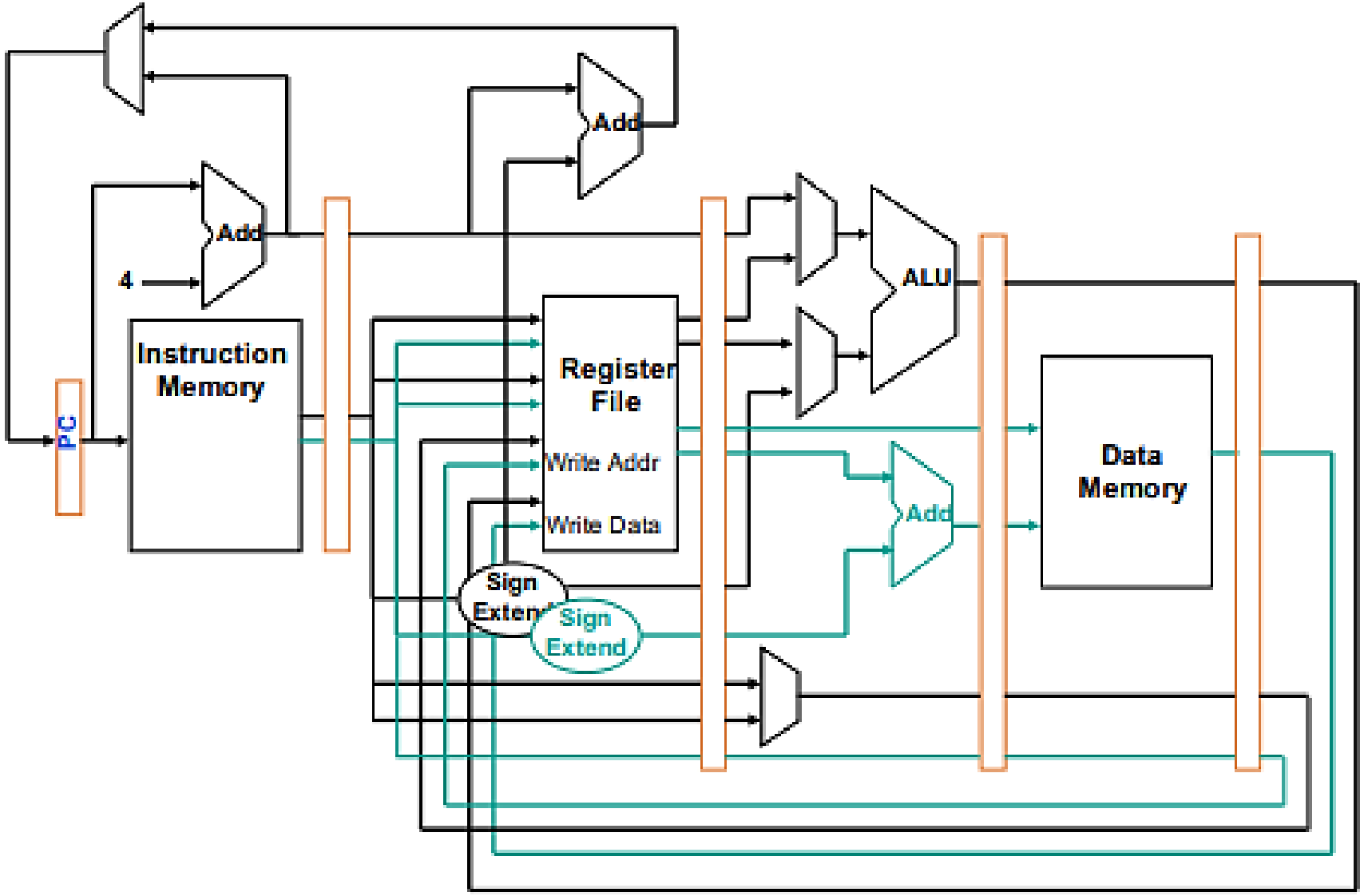
2 Issue RISC

Consider a 2-issue RISC-V with a 2 instr bundle



- Instructions are always fetched, decoded, and issued in pairs
- If one instr of the pair can not be used, it is replaced with a noop
- Need 4 read ports and 2 write ports and a separate memory address adder

2 Issue Datapath



VLIW

Basic VLIW Approach

The Basic VLIW Approach

VLIWs use multiple, independent functional units. Rather than attempting to issue multiple, independent instructions to the units, a VLIW packages the multiple operations into one very long instruction or requires that the instructions in the issue packet satisfy the same constraints. Because there is no fundamental difference in the two approaches, we will just assume that multiple operations are placed in one instruction, as in the original VLIW approach.

Because the advantage of a VLIW increases as the maximum issue rate grows, we focus on a wider issue processor. Indeed, for simple two-issue processors, the overhead of a superscalar is probably minimal. Many designers would probably argue that a four-issue processor has manageable overhead, but as we will see later in this chapter, the growth in overhead is a major factor limiting wider issue processors.

Very Long Instruction Word

- **Each "instruction" has explicit coding for multiple operations**
  - In IA-64, grouping called a "packet"
  - In Transmeta, grouping called a "molecule" (with "atoms" as ops)
- **Tradeoff instruction space for simple decoding**
  - The long instruction word has room for many operations
  - By definition, all the operations the compiler puts in the long instruction word are independent => execute in parallel
  - E.g., 1 integer operation/branch, 2 FP ops, 2 Memory refs
    - » 16 to 24 bits per field => 5*16 or 80 bits to 5*24 or 120 bits wide
  - Need compiling technique that schedules across several branches

Parallelism needed for VLIW

Let's consider a VLIW processor with instructions that contain five operations, including one integer operation (which could also be a branch), two floating-point operations, and two memory references. The instruction would have a set of fields for each functional unit—perhaps 16–24 bits per unit, yielding an instruction length of between 80 and 120 bits. By comparison, the Intel Itanium 1 and 2 contain six operations per instruction packet (i.e., they allow concurrent issue of two three-instruction bundles, as Appendix H describes).

To keep the functional units busy, there must be enough parallelism in a code sequence to fill the available operation slots. This parallelism is uncovered by unrolling loops and scheduling the code within the single larger loop body. If the unrolling generates straight-line code, then *local scheduling* techniques, which operate on a single basic block, can be used. If finding and exploiting the parallelism require scheduling code across branches, a substantially more complex *global scheduling* algorithm must be used. Global scheduling algorithms are not only more complex in structure, but they also must deal with significantly more complicated trade-offs in optimization, because moving code across branches is expensive.

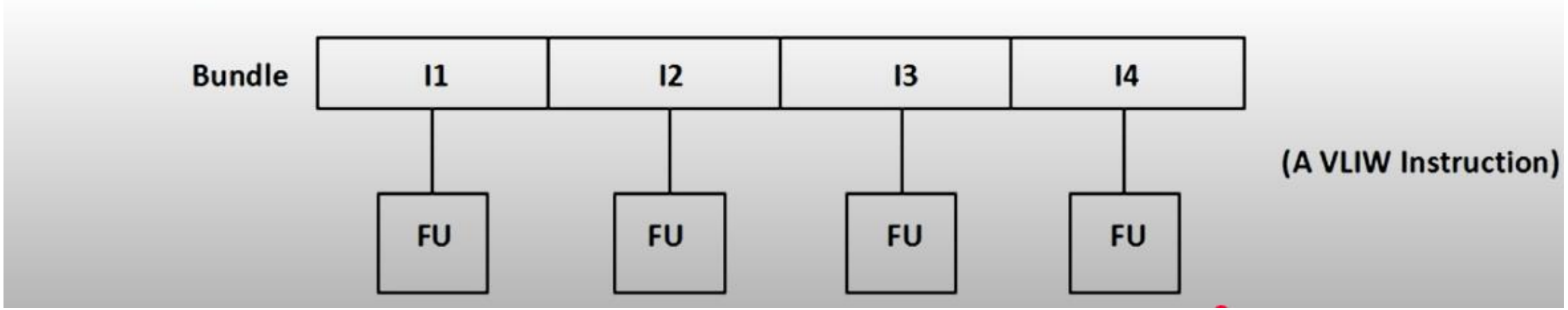
VLIW Processor

VLIW is also an architectural design that exploit instruction level parallelism.

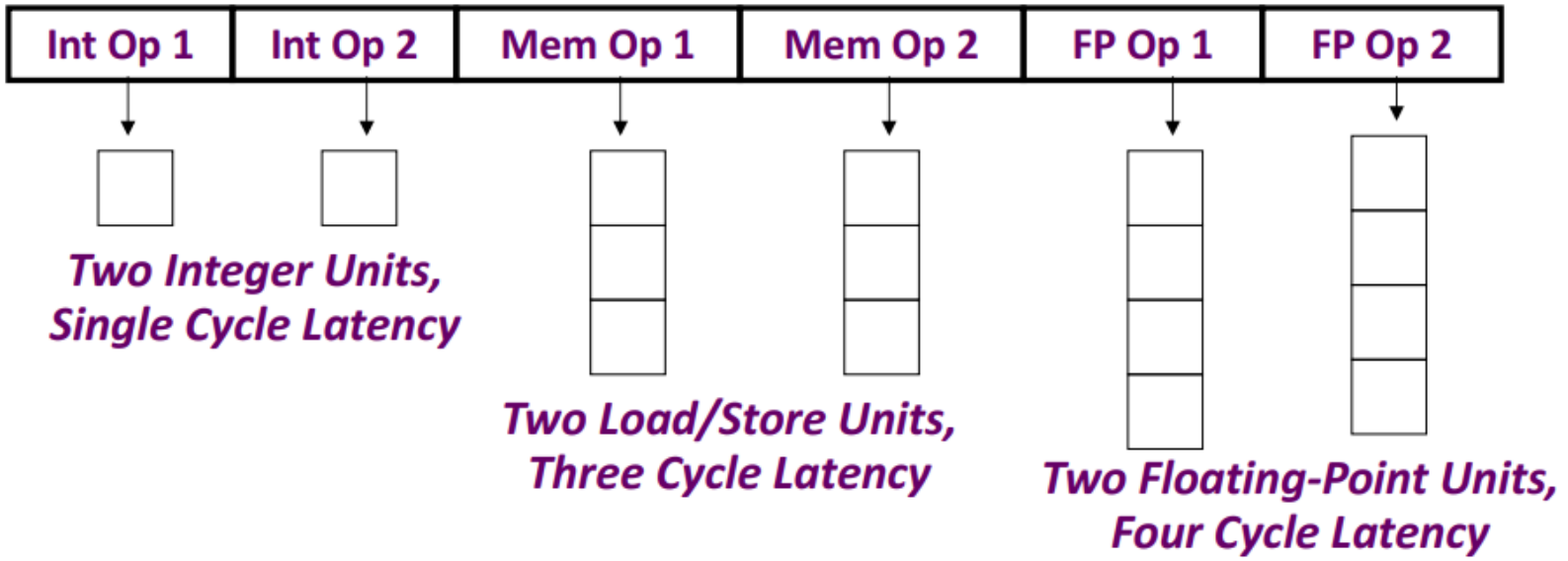
The VLIW processor rely on compile time analysis to identify and bundle together instructions that can be executed concurrently.

Compiler is responsible for static scheduling of instructions in VLIW processor. Compiler finds out which operations can be executed in parallel in the program and groups together these operations in single instruction known as a bundle or an issue packet which is the very large instruction word.Compiler ensures that an operation is not issued before its operands are ready.

They deploy multiple independent functional units.

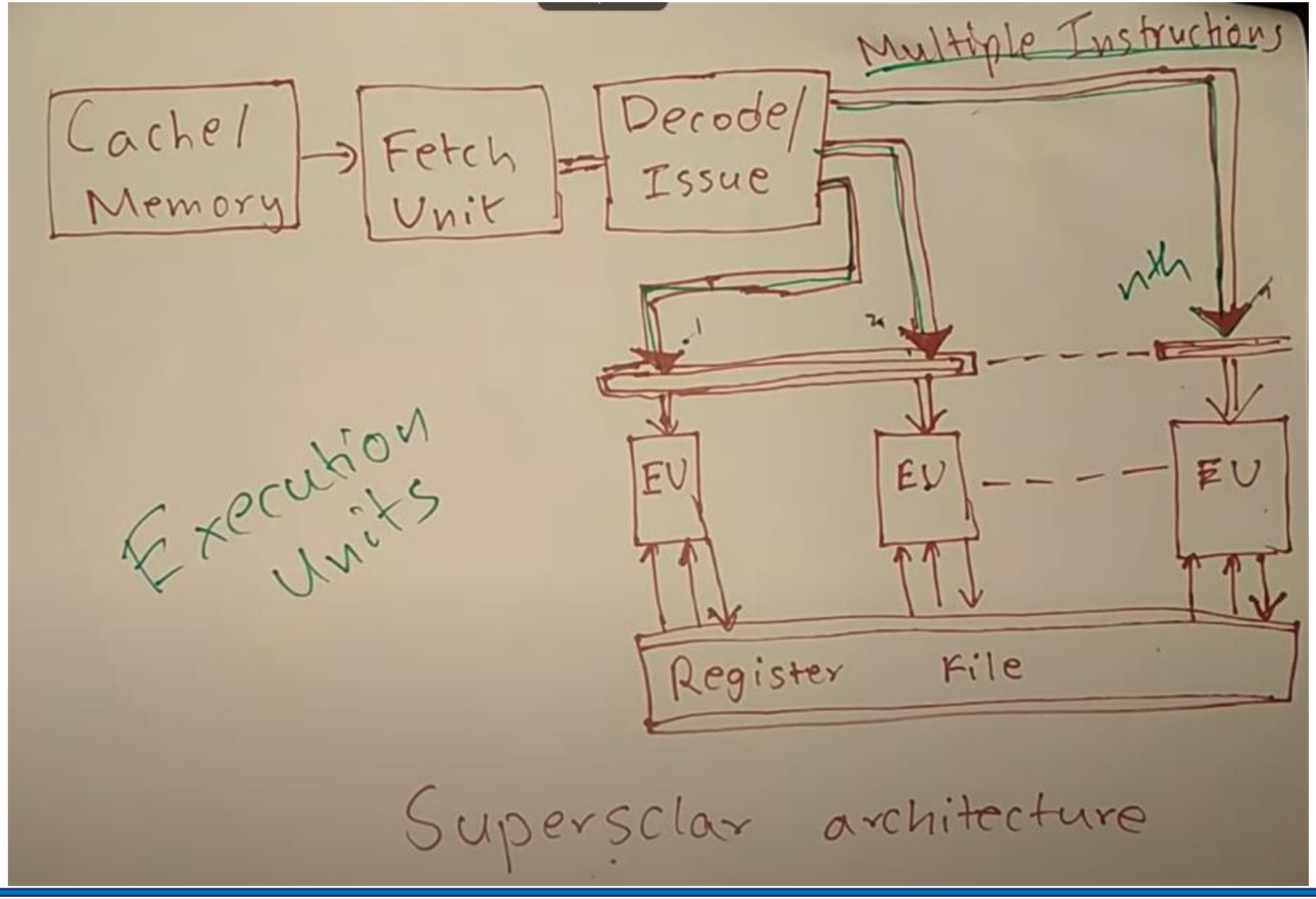


Form VLIW Instruction Word

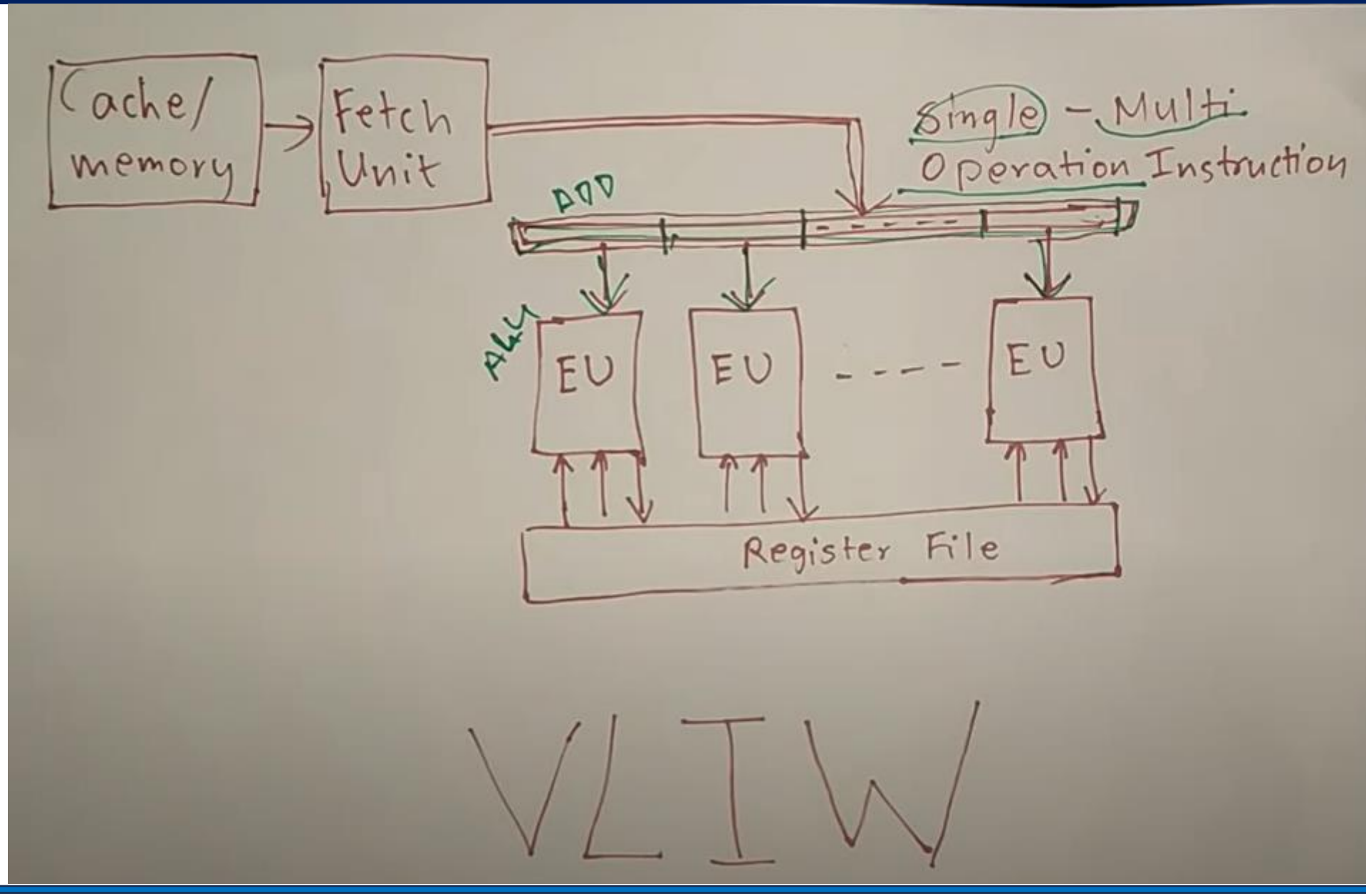


- Multiple operations packed into one instruction
- Each operation slot is for a fixed function
- Constant operation latencies are specified
- Architecture requires guarantee of:
  - Parallelism within an instruction => no cross-operation RAW check
  - No data use before data ready => no data interlocks

Superscalar Architecture



VLIW Architecture



Compare VLIW and Superscalar

VLIW Processors	Superscalar Processors
Parallelism is performed statically at compile time by the compiler.	Parallelism is performed dynamically at run time by hardware.
Data dependency is checked by compiler.	Data dependency is checked and resolved by hardware.
Less parallelism can be exploited.	More parallelism can be exploited.
Simple hardware and complex compiler.	Simple compiler and complex hardware.

Problems in VLIW Approach

For the original VLIW model, there were both technical and logistical problems that made the approach less efficient. The technical problems were the increase in code size and the limitations of the lockstep operation. Two different elements combine to increase code size substantially for a VLIW. First, generating enough operations in a straight-line code fragment requires ambitiously unrolling loops (as in earlier examples), thereby increasing code size. Second, whenever instructions are not full, the unused functional units translate to wasted bits in the instruction encoding. In Appendix H, we examine software scheduling approaches, such as software pipelining, that can achieve the benefits of unrolling without as much code expansion.

To combat this code size increase, clever encodings are sometimes used. For example, there may be only one large immediate field for use by any functional unit. Another technique is to compress the instructions in main memory and expand them when they are read into the cache or are decoded. In Appendix H, we show

Hazard Detection in VLIW, Code Compatibility

Early VLIWs operated in lockstep; there was no hazard-detection hardware at all. This structure dictated that a stall in any functional unit pipeline must cause the entire processor to stall because all the functional units had to be kept synchronized. Although a compiler might have been able to schedule the deterministic functional units to prevent stalls, predicting which data accesses would encounter a cache stall and scheduling them were very difficult to do. Thus caches needed to be blocking and causing *all* the functional units to stall. As the issue rate and number of memory references became large, this synchronization restriction became unacceptable. In more recent processors, the functional units operate more independently, and the compiler is used to avoid hazards at issue time, while hardware checks allow for unsynchronized execution once instructions are issued.

Binary code compatibility has also been a major logistical problem for general-purpose VLIWs or those that run third-party software. In a strict VLIW approach, the code sequence makes use of both the instruction set definition and the detailed pipeline structure, including both functional units and their latencies. Thus different numbers of functional units and unit latencies require different versions of the code. This requirement makes migrating between successive implementations, or between implementations with different issue widths, more difficult than it is for a superscalar design. Of course, obtaining improved performance from a new superscalar design may require recompilation. Nonetheless, the ability to run old binary files is a practical advantage for the superscalar approach. In the domain-specific

Example page 220 (come back later after examples)

Example Suppose we have a VLIW that could issue two memory references, two FP operations, and one integer operation or branch in every clock cycle. Show an unrolled version of the loop `x[i] = x[i] + s` (see page 158 for the RISC-V code) for such a processor. Unroll as many times as necessary to eliminate any stalls.

Solution (come back later)

Answer Figure 3.20 shows the code. The loop has been unrolled to make seven copies of the body, which eliminates all stalls (i.e., completely empty issue cycles), and runs in 9 cycles for the unrolled and scheduled loop. This code yields a running rate of seven results in 9 cycles, or 1.29 cycles per result, nearly twice as fast as the two-issue superscalar of Section 3.2 that used unrolled and scheduled code.

Memory reference 1	Memory reference 2	FP operation 1	FP operation 2	Integer operation/branch
`fld f0,0(x1)`	`fld f6,-8(x1)`			
`fld f10,-16(x1)`	`fld f14,-24(x1)`			
`fld f18,-32(x1)`	`fld f22,-40(x1)`	`fadd.d f4,f0,f2`	`fadd.d f8,f6,f2`	
`fld f26,-48(x1)`		`fadd.d f12,f0,f2`	`fadd.d f16,f14,f2`	
		`fadd.d f20,f18,f2`	`fadd.d f24,f22,f2`	
`fsd f4,0(x1)`	`fsd f8,-8(x1)`	`fadd.d f28,f26,f24`		
`fsd f12,-16(x1)`	`fsd f16,-24(x1)`			`addi x1,x1,-56`
`fsd f20,24(x1)`	`fsd f24,16(x1)`			
`fsd f28,8(x1)`				`bne x1,x2,Loop`

Figure 3.20 VLIW instructions that occupy the inner loop and replace the unrolled sequence. This code takes 9 cycles assuming correct branch prediction. The issue rate is 23 operations in 9 clock cycles, or 2.5 operations per cycle. The efficiency, the percentage of available slots that contained an operation, is about 60%. To achieve this issue rate requires a larger number of registers than RISC-V would normally use in this loop. The preceding VLIW code sequence requires at least eight FP registers, whereas the same code sequence for the base RISC-V processor can use as few as two FP registers or as many as five when unrolled and scheduled.

Branch Target Buffer (Cache)

Branch Target Buffers

To reduce the branch penalty for our simple five-stage pipeline, as well as for deeper pipelines, we must know whether the as-yet-undecoded instruction is a branch and, if so, what the next program counter (PC) should be. If the instruction is a branch and we know what the next PC should be, we can have a branch penalty of zero. A branch-prediction cache that stores the predicted address for the next instruction after a branch is called a *branch-target buffer* or *branch-target cache*. Figure 3.25 shows a branch-target buffer.

Because a branch-target buffer predicts the next instruction address and will send it out *before* decoding the instruction, we *must* know whether the fetched instruction is predicted as a taken branch. If the PC of the fetched instruction matches an address in the prediction buffer, then the corresponding predicted PC is used as the next PC. The hardware for this branch-target buffer is essentially identical to the hardware for a cache.

If a matching entry is found in the branch-target buffer, fetching begins immediately at the predicted PC. Note that unlike a branch-prediction buffer, the predictive entry must be matched to this instruction because the predicted PC will be sent out before it is known whether this instruction is even a branch. If the processor did not check whether the entry matched this PC, then the wrong PC would be sent out for instructions that were not branches, resulting in worse performance. We need to store only the predicted-taken branches in the branch-target buffer because an untaken branch should simply fetch the next sequential instruction, as if it were not a branch.

Branch Target Buffer Architecture

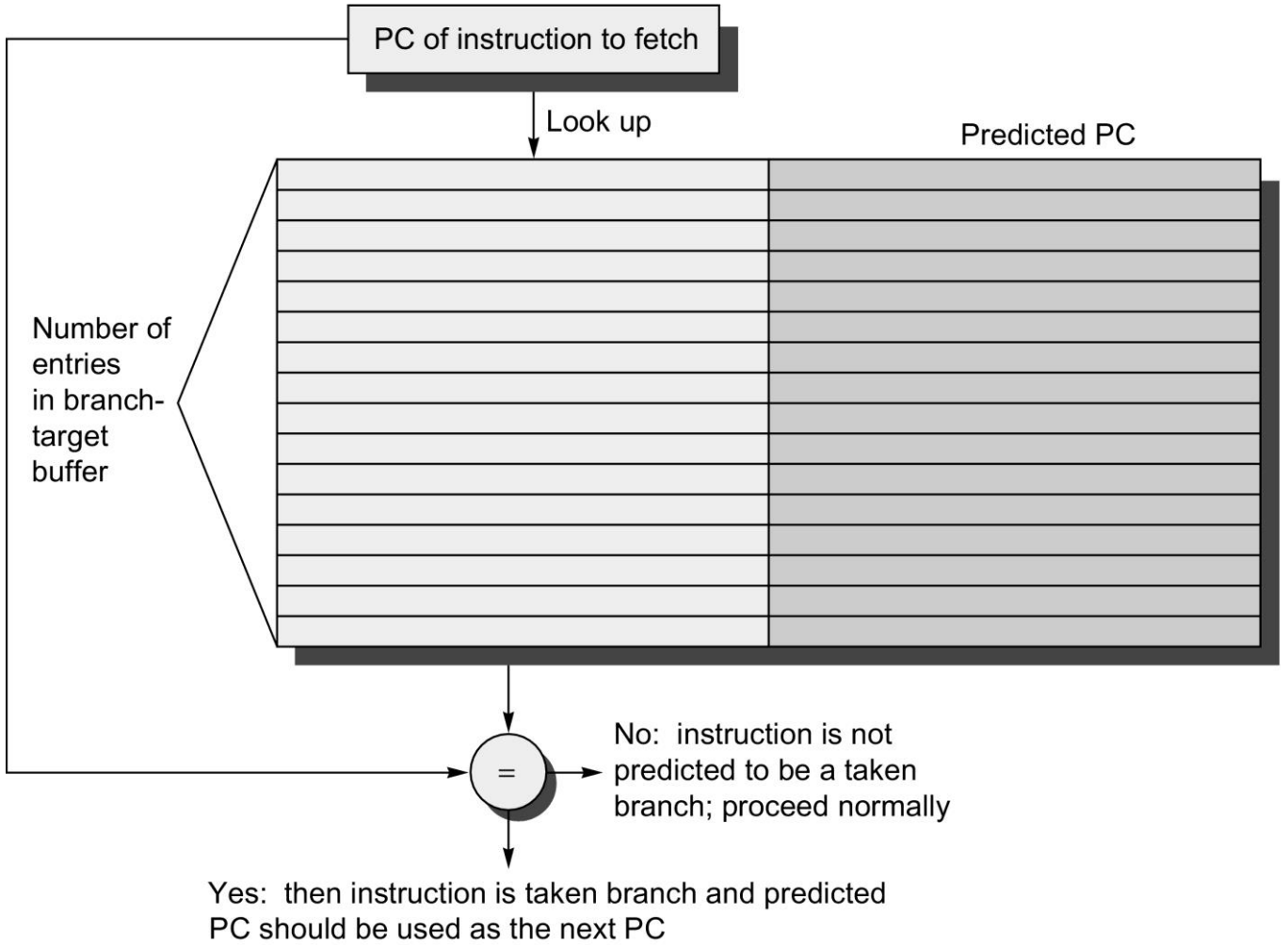


Figure 3.25 A branch-target buffer. The PC of the instruction being fetched is matched against a set of instruction addresses stored in the first column; these represent the addresses of known branches. If the PC matches one of these entries, then the instruction being fetched is a taken branch, and the second field, predicted PC, contains the prediction for the next PC after the branch. Fetching begins immediately at that address. The third field, which is optional, may be used for extra prediction state bits.

3 Steps in BTB

for instructions that were not branches, resulting in worse performance. We need to store only the predicted-taken branches in the branch-target buffer because an untaken branch should simply fetch the next sequential instruction, as if it were not a branch.

Figure 3.26 shows the steps when using a branch-target buffer for a simple five-stage pipeline. As we can see in this figure, there will be no branch delay if a branch-prediction entry is found in the buffer and the prediction is correct. Otherwise, there will be a penalty of at least two clock cycles. Dealing with the mispredictions and misses is a significant challenge because we typically will have to halt instruction fetch while we rewrite the buffer entry. Thus we want to make this process fast to minimize the penalty.

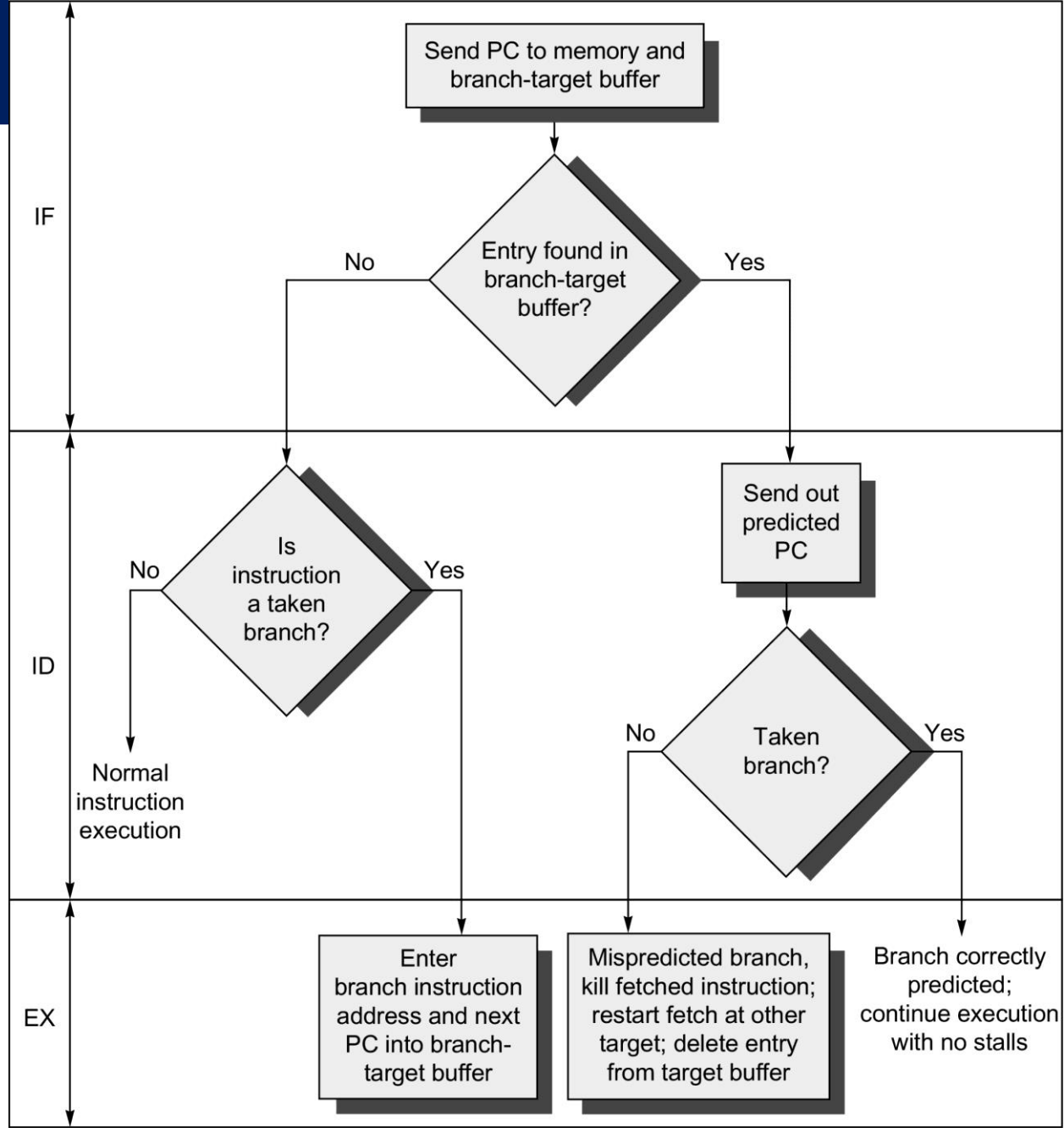


Figure 3.26 The steps involved in handling an instruction with a branch-target buffer.

Example page 230 (come back later)

Instruction in buffer	Prediction	Actual branch	Penalty cycles
Yes	Taken	Taken	0
Yes	Taken	Not taken	2
No		Taken	2
No		Not taken	0

Figure 3.27 Penalties for all possible combinations of whether the branch is in the buffer and what it actually does, assuming we store only taken branches in the buffer. There is no branch penalty if everything is correctly predicted and the branch is found in the target buffer. If the branch is not correctly predicted, the penalty is equal to 1 clock cycle to update the buffer with the correct information (during which an instruction cannot be fetched) and 1 clock cycle, if needed, to restart fetching the next correct instruction for the branch. If the branch is not found and taken, a 2-cycle penalty is encountered, during which time the buffer is updated.

Example Determine the total branch penalty for a branch-target buffer assuming the penalty cycles for individual mispredictions in Figure 3.27. Make the following assumptions about the prediction accuracy and hit rate:

- Prediction accuracy is 90% (for instructions in the buffer).
- Hit rate in the buffer is 90% (for branches predicted taken).

Answer We compute the penalty by looking at the probability of two events: the branch is predicted taken but ends up being not taken, and the branch is taken but is not found in the buffer. Both carry a penalty of two cycles.

$$\begin{aligned}\text{Probability (branch in buffer, but actually not taken)} &= \text{Percent buffer hit rate} \\ &\quad \times \text{Percent incorrect predictions} \\ &= 90\% \times 10\% = 0.09\end{aligned}$$

$$\text{Probability (branch not in buffer, but actually taken)} = 10\%$$

$$\text{Branch penalty} = (0.09 + 0.10) \times 2$$

$$\text{Branch penalty} = 0.38$$

The improvement from dynamic branch prediction will grow as the pipeline length, and thus the branch delay grows; in addition, better predictors will yield a greater performance advantage. Modern high-performance processors have branch misprediction delays on the order of 15 clock cycles; clearly, accurate prediction is critical!